

# 08 · Multicore-Synchronisation & Safety

HSEM, Watchdogs, MPU · EZS · DHBW Stuttgart

Prof. Dr.-Ing. Daniel Klünder

Folie

1 / 51

## Willkommen in der Welt der Robustheit!

- Herzlich Willkommen zum 8. Termin unserer Vorlesung.
- Letzte Woche: Timing-Analyse mit der BlueBox.
- Davor (Termin 6): Unser erstes Multicore-Projekt mit Shared Memory und Cache-Kohärenz.
- Heute lösen wir das letzte große architektonische Multicore-Problem.
- Und wir widmen uns dem Thema **Safety** (Ausfallsicherheit).
- Ziel: Wir bauen Systeme, die sich selbst überwachen und im Notfall retten können!

## Notizen

Guten Morgen und herzlich willkommen zur achten Vorlesung! In den vergangenen Wochen haben Sie gelernt, wie man Systeme programmiert und wie man beweist, dass sie hartes Echtzeit-Timing einhalten. Sie haben zwei Kerne miteinander kommunizieren lassen und das gefährliche Cache-Problem gelöst. Aber wenn wir ehrlich sind: Unser Code von damals war naiv. Wir haben uns darauf verlassen, dass immer nur ein Kern zur selben Zeit in das geteilte RAM-Strukt schreibt. Was aber, wenn beide das gleichzeitig tun? Heute lösen wir dieses Problem auf Silizium-Ebene. Im zweiten Teil der Vorlesung beschäftigen wir uns mit Safety. Wir klären die Frage: Wer rettet eigentlich das Auto, wenn Ihre Software trotz all unserer Bemühungen in einen Deadlock rennt?

Folie

2 / 51

## Agenda für heute

- Recap: Unser Multicore-Handshake (Ping-Pong).
- Das ungelöste Problem: Multicore Race Conditions.
- Hardware Semaphores (HSEM): Atomarität auf Bus-Ebene.
- Safety in Embedded Systems: IWDG und WWDG (Fenster, EWI).
- Speicherschutz mit der MPU (Null-Pointer, Stack Guard).
- Labor: HSEM-Ping-Pong umbauen und WWDG live crashen.

## Notizen

Unsere heutige Agenda teilt sich in zwei große Blöcke. Zuerst werfen wir einen Blick zurück auf unser Multicore-Labor. Wir zeigen auf, warum unser Ping-Pong-Protokoll zwar funktionierte, für asynchrone Sensordaten aber völlig ungeeignet ist. Wir führen das HSEM-Modul unseres Chips ein, um Hardware-Schlösser über Core-Grenzen hinweg zu bauen. Im zweiten Block wechseln wir zur Safety: Independent und Window Watchdog inklusive Early Wakeup Interrupt, danach die Memory Protection Unit als physikalisches Sicherheitsnetz. Zum Abschluss bereiten wir das Labor vor: HSEM-Synchronisation und WWDG-Crash-Tests.

## Folie

3/51

### Recap: Der Ping-Pong Datenaustausch

- In Woche 6 haben wir Daten über den SRAM4 ausgetauscht.
- Wir nutzten ein strenges Handshake-Protokoll:

```
typedef struct {  
    uint8_t m7_ready;  
    uint8_t m4_ready;  
    uint32_t payload;  
} SharedData_t;
```

- Der Ablauf: Der M7 schreibt den Payload, setzt `m7_ready = 1` und wartet.
- Der M4 liest, schreibt und setzt `m4_ready = 1`.
- Das System war strikt synchron. Es gab keine Kollisionen.

## Notizen

Erinnern wir uns an die Struktur unseres Shared Memorys. Wir hatten ein C-Struct im SRAM4 abgelegt, der Cache war per MPU deaktiviert. Wir haben die Kommunikation über zwei Flags orchestriert: `m7_ready` und `m4_ready`. Es war wie beim Funkverkehr: Ein Kern sprach, schloss mit "Over", und erst dann durfte der andere Kern antworten. Dieses strikt synchrone Handshake-Protokoll verhinderte, dass jemals beide Kerne in exakt derselben Mikrosekunde auf den payload-Wert schreibend zugreifen wollten.

## Das ungelöste Problem

- Was passiert in der Realität?
- Beide Kerne arbeiten völlig asynchron.
- Der Cortex-M7 (Ethernet-Stack) und der Cortex-M4 (Motor-Regler) wollen gleichzeitig auf einen globalen Fehler-Counter im SRAM4 zugreifen.
- Wir wollen `shared_data.error_count++` ausführen.
- Beide Kerne rufen (ohne Handshake) diesen Befehl auf.
- Ergebnis: Datenkorruption.

## Notizen

In einem echten Steuergerät läuft das aber meist asynchron ab. Stellen Sie sich vor, wir speichern in unserem Shared Memory einen globalen System-Fehlerzähler. Der Cortex-M7 detektiert einen fehlerhaften Netzwerk-Frame und möchte den Zähler erhöhen. In exakt derselben Nanosekunde detektiert der Cortex-M4 eine Überspannung am Motor und möchte ebenfalls diesen globalen Zähler erhöhen. Es gibt kein "Over"-Flag. Beide greifen stumpf und asynchron auf den Speicher zu. Das Ergebnis ist eine Multicore Race Condition.

## Warum ein FreeRTOS-Mutex hier versagt

- „Wir wissen doch, wie man das löst! Wir nehmen einen Mutex!“
- Falsch! Ein FreeRTOS-Mutex (`osMutexAcquire`) funktioniert hier nicht.
- Warum?
  - Der Cortex-M7 läuft bei uns Bare-Metal (er kennt die FreeRTOS-API gar nicht).
  - Selbst wenn beide Kerne FreeRTOS nutzen würden: Jeder Kern hat sein eigenes, isoliertes OS im RAM!
  - Der Scheduler des M4 hat keine physikalische Kontrolle über die Register des M7.
- Ein reines Software-Schloss reicht bei Multicore nicht aus.

## Notizen

Sie werden jetzt sofort denken: "Halt, das Problem kenne ich doch aus Vorlesung 4! Das ist eine Race Condition. Dafür haben wir doch den Mutex gelernt!". Das ist ein gefährlicher Trugschluss. Ein FreeRTOS-Mutex ist ein Software-Konstrukt. Er ist nichts weiter als eine kleine Variable im lokalen RAM des Cortex-M4, verwaltet vom lokalen Scheduler des M4. Der Cortex-M7 läuft aber Bare-Metal. Er kennt die C-Funktion `osMutexAcquire` überhaupt nicht. Und selbst wenn wir auf beiden Kernen FreeRTOS installieren würden: Jedes OS ist isoliert. Der Scheduler von Core 2 kann Core 1 nicht schlafen legen. Ein Software-Schloss ist für Multicore-Systeme völlig nutzlos, da sich die Kerne logisch nicht gegenseitig blockieren können.

## Die Multicore Race Condition

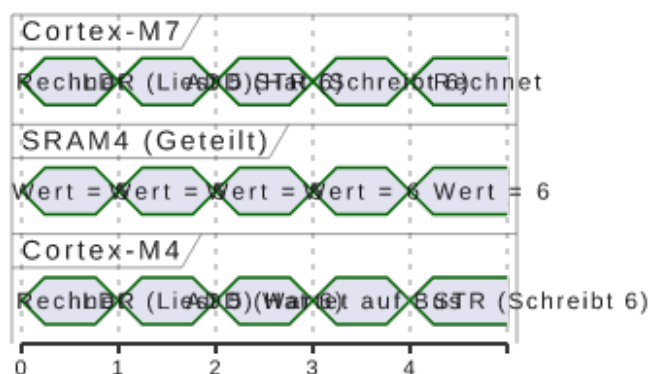
- Wie sieht die Zerstörung auf dem Bus aus?
- Zähler steht auf 5 (im SRAM4).
- Der M7 liest die 5 in sein CPU-Register und addiert zu 6.
- Der M4 liest (1 Taktzyklus später) ebenfalls die 5 in sein Register und addiert zu 6.
- Der M7 schreibt die 6 in den SRAM4.
- Der M4 schreibt seine 6 in den SRAM4.
- Zwei Fehler aufgetreten, aber der Zähler stieg nur um 1!

### Notizen

Schauen wir uns das auf Hardware-Ebene an. Wir wissen, dass ein counter++ Befehl aus Load, Add und Store besteht. Bei Single-Core war die Präemption durch Interrupts das Problem. Bei Multicore ist es noch drastischer: Die Befehle laufen wirklich in physikalischer Gleichzeitigkeit ab! Der M7 lädt die 5 aus dem SRAM über die Bus-Matrix. Fast im selben Taktzyklus sendet auch der M4 einen Lese-Befehl an die Matrix. Die Matrix schickt beiden die 5. Beide Kerne addieren intern in ihrer ALU eine 1. Und dann schicken beide Kerne einen Schreib-Befehl für die 6 an die Bus-Matrix. Der Zähler steht auf 6, ein kompletter Fehler wurde verschluckt.

### Folie

## Visualisierung: Multicore-Zerstörung



- Die AXI-Bus-Matrix regelt zwar den Stau beim Schreiben (Takt 3 und 4), aber der logische Schaden ist bereits angerichtet!

### Notizen

Hier sehen Sie dieses Drama im Timing-Diagramm. Die Bus-Matrix des STM32H755 verhindert zwar, dass Signale elektrisch kollidieren. Bei T=3 will der M7 schreiben, und der M4 muss kurz warten. Die

Matrix reiht die Schreibzugriffe brav hintereinander auf. Das Problem ist nur: Der logische Schaden ist schon passiert, weil beide Cores bei  $T=1$  denselben alten Wert ausgelesen haben. Wir brauchen einen Mechanismus, der verhindert, dass Core 2 den Wert überhaupt ausliest, solange Core 1 seinen Dreisatz aus Load, Add und Store nicht komplett abgeschlossen hat.

## Folie

8/51

### Die Anforderung: Ein Hardware-Schloss

- Wir brauchen ein Schloss, das:
  - Für beide Kerne physikalisch sichtbar ist.
  - Auf Hardware-Ebene atomar arbeitet.
  - Eine Lese- und Schreib-Operation in einem einzigen Bus-Takt garantiert (Test-and-Set).
- Die Antwort von ARM / STMicroelectronics: Das HSEM-Modul.

## Notizen

Die Lösung kann also nicht in der Software liegen. Wir benötigen ein Schloss, das in das Silizium gebrannt ist. Ein Schloss, auf das beide Kerne über den Bus zugreifen können, und das hardwareseitig garantiert, dass immer nur exakt ein Kern den Schlüssel bekommt, selbst wenn beide Signale auf die Nanosekunde genau ankommen. Dafür hat die Halbleiterindustrie spezielle Schaltungen entwickelt, sogenannte "Test-and-Set" Register, die eine Lese- und Schreiboperation atomar in einem unteilbaren Bus-Zyklus durchführen. Auf unserem Board heißt dieses Modul HSEM.

## Folie

9/51

### Einführung in Hardware Semaphores (HSEM)

- HSEM = Hardware Semaphore.
- Ein dediziertes Hardware-Modul im STM32H755.
- Es bietet 32 unabhängige, physikalische Schlösser (HSEM 0 bis HSEM 31).
- Jeder Kern kann jedes dieser 32 Schlösser sperren oder freigeben.
- Wichtig: HSEM ist konzeptionell ein Mutex (mit Ownership), obwohl „Semaphore“ im Namen steht!

## Notizen

HSEM steht für Hardware Semaphore. Es ist ein eigener Peripherie-Block auf dem Die unseres Mikrocontrollers, ähnlich wie ein UART- oder Timer-Modul. Es stellt uns 32 völlig unabhängige physikalische Schlösser zur Verfügung, nummeriert von 0 bis 31. Wir könnten also 32 verschiedene Shared-Memory-Variablen unabhängig voneinander absichern. Lassen Sie sich vom Namen "Semaphore" nicht verwirren: Ein Hardware-Semaphor auf dem STM32 funktioniert konzeptionell wie ein Mutex. Es merkt sich ganz

genau, welcher Kern ihn abgeschlossen hat, und nur dieser Kern darf ihn wieder aufsperrern.

Folie

10/51

## HSEM Architektur im STM32H755

- Wo liegt das HSEM-Modul?
- Es sitzt zentral am AHB4-Bus und ist für den M7 und den M4 gleichwertig erreichbar.
- Der 2-Step-Lock:
  - Wenn ein Kern das Schloss sperrt, trägt die Hardware die Core ID (z. B. M4) in das HSEM-Register ein.
  - Gleichzeitig wird eine Process ID eingetragen (z. B. welche FreeRTOS-Task den Lock angefordert hat).
- Das Schloss gehört nun diesem spezifischen Task auf diesem spezifischen Kern!

### Notizen

Das HSEM-Modul sitzt strategisch günstig an einem zentralen Bus-Knoten, den beide Kerne gut erreichen können. Wenn der Cortex-M4 ein Schloss anfordert, liest die HSEM-Hardware über den Bus die ID des Kerns aus und brennt diese sogenannte Core ID in das Schloss-Register ein. Zusätzlich können wir aus der Software noch eine Process ID mitgeben. Wir können also sagen: "Dieses Schloss gehört dem Cortex-M4, und zwar speziell der Task 5". Die Hardware verriegelt nun dieses Register. Versucht der Cortex-M7 daraufhin abzuschließen, weist die Hardware den Befehl sofort ab, weil die Core ID nicht passt.

Folie

11/51

## Atomarität auf Silizium-Ebene

- Was passiert, wenn M7 und M4 exakt in derselben Nanosekunde den HSEM-Sperrbefehl senden?
- Die Bus-Matrix-Arbiter-Magie:
  - Die Bus-Matrix hat einen Hardware-Arbiter (Schiedsrichter).
  - Dieser Arbiter erzwingt physikalisch, dass ein Zugriffs-Request immer um mindestens einen Taktzyklus verzögert wird.
- Der erste Request, der durchkommt, führt ein atomares „Read-Lock“ aus.
- Der zweite Request liest bereits den gesperrten Zustand aus.
- Auf Hardware-Ebene gibt es keine Gleichzeitigkeit!

### Notizen

Was aber, wenn wirklich beide Kerne auf die Nanosekunde genau den Sperr-Befehl für Schloss Null abschicken? Hier greift die Magie des Bus-Matrix-Arbitors (HSEM hängt an AHB4). Ein Schiedsrichter in

Silizium. Auf digitaler Gatter-Ebene ist physikalische Gleichzeitigkeit ausgeschlossen. Der Arbiter wird eines der beiden Signale hardwareseitig verwerfen oder um einen winzigen Taktzyklus verzögern. Das Signal, das gewinnt, führt eine atomare Read-Lock-Operation im HSEM-Modul aus. Wenn das Signal des zweiten Kerns den Bruchteil einer Nanosekunde später ankommt, ist der Schalter im Flip-Flop des Registers bereits umgelegt, und der zweite Kern bekommt eine Abweisung zurück. Wir haben absolute Sicherheit.

Folie

12 / 51

## Die HAL-API für HSEM

- Sperren (Acquire)
  - `HAL_HSEM_FastTake (HSEM_ID)`
  - Liefert `HAL_OK`, wenn wir das Schloss bekommen haben.
  - Liefert `HAL_ERROR`, wenn der andere Kern (oder eine andere Task) es bereits besitzt.
- Entsperren (Release)
  - `HAL_HSEM_Release (HSEM_ID, ProcessID)`
  - Gibt das Schloss frei, aber nur, wenn wir der rechtmäßige Besitzer sind!
- Achtung: `FastTake` blockiert nicht! Es liefert sofort das Ergebnis (Try-Lock).

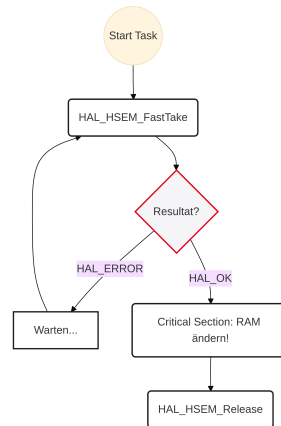
## Notizen

Wie binden wir das in unseren C-Code ein? STMicroelectronics liefert uns dafür zwei extrem schnelle HAL-Funktionen. Mit `HAL_HSEM_FastTake` greifen wir nach dem Schloss. Wir übergeben nur die ID, zum Beispiel 0 für Schloss Null. Ganz wichtig: Diese Funktion ist kein RTOS-Mutex! Sie blockiert nicht. Sie ist ein sogenannter "Try-Lock". Sie greift nach dem Schloss und liefert sofort, ohne jede Verzögerung, ein Ergebnis zurück. Entweder `HAL_OK`, dann gehört das Schloss uns. Oder `HAL_ERROR`, dann gehört es jemand anderem. Mit `HAL_HSEM_Release` geben wir das Schloss wieder frei.

Folie

13 / 51

## Flowchart: HSEM Take



- Wenn wir `HAL_ERROR` bekommen, müssen wir die Anforderung wiederholen.

### Notizen

Die Logik in unserem C-Code sieht also wie folgt aus: Wir fordern das Schloss mit `FastTake` an. Wir prüfen das Resultat. Wenn wir einen Error bekommen, ist der Bus belegt. Wir müssen warten und es später noch einmal versuchen. Wenn wir ein OK bekommen, dürfen wir endlich unsere `counter++` Operation auf dem Shared Memory im SRAM4 ausführen. Danach müssen wir zwingend den HSEM-Release aufrufen, damit der andere Kern weiterarbeiten kann. Die große Frage, die sich hier stellt, ist: Wie implementieren wir den Kasten "Warten..." auf eine Weise, die unser Betriebssystem nicht zerstört?

### Folie

14 / 51

## Warten auf den HSEM: Polling vs. Interrupts

- Wir erinnern uns: Spinlocks (`while (take != OK)`) verbrennen CPU-Zeit.
- Bare-Metal (M7):
  - Hier können wir pollen. Eine kurze `while`-Schleife um den `FastTake` ist vertretbar, wenn die Critical Section des M4 sehr kurz ist.
- FreeRTOS (M4):
  - Polling zerstört unsere Task-Auslastung!
  - Wir müssen das OS nutzen (`osWaitForever`), aber `FastTake` hat kein Timeout!
- Wie lösen wir das? Mit HSEM-Interrupts!

### Notizen

Das Warten ist das Kernproblem. Wenn wir im Cortex-M7 Bare-Metal Code sind, können wir einfach eine schnelle `While`-Schleife um das `FastTake` bauen. Da der M7 kein OS hat, verliert er keine Task-Prioritäten, er verbrennt nur ein paar Taktzyklen Strom. Auf dem Cortex-M4 haben wir aber FreeRTOS laufen! Eine `While`-Schleife blockiert hier den Kernel: die Task bleibt *Running* und verbraucht CPU — Busy-Wait / potenzielle Starvation niederpriorer Tasks, nicht die klassische Priority Inversion (die braucht einen Mutex).

Wir müssen unsere RTOS-Task zwingend in den “Blocked”-Zustand schlafen schicken, während wir auf den M7 warten. Aber FastTake hat keinen Timeout-Parameter, es schläft nicht!

Folie

15/51

### HSEM Interrupts: M7 weckt M4-Task

- Das HSEM-Modul kann Hardware-Interrupts auslösen!
- Der smarte Workflow für den M4:
  1. M4 ruft `FastTake` auf → Besetzt (Fehler).
  2. M4 aktiviert den HSEM-Interrupt für dieses Schloss und legt seine FreeRTOS-Task über einen Binären Semaphor schlafen (`osWaitForever`).
  3. M7 rechnet fertig und ruft `Release` auf.
  4. Hardware löst einen Interrupt auf dem M4 aus!
  5. Die ISR des M4 gibt den Binären Semaphor frei → Die M4-Task wird vom OS blitzschnell geweckt und holt sich das Hardware-Schloss!

### Notizen

Die Lösung ist architektonisch brillant: Das HSEM-Modul kann Hardware-Interrupts über Bus-Grenzen hinweg abfeuern. Der Workflow für unser FreeRTOS auf dem M4 sieht so aus: Wir rufen `FastTake` auf. Es schlägt fehl. Also weisen wir die HSEM-Hardware an: “Bitte schick mir einen Interrupt, sobald dieses Schloss entriegelt wird!”. Dann rufen wir die CMSIS-API auf und legen unsere Task per Software-Semaphor in den “Blocked”-Zustand. Die CPU Auslastung fällt auf 0. Sobald der M7 den Release-Befehl ausführt, feuert die Hardware einen Interrupt auf den M4. Unsere kleine ISR fängt den Interrupt ab, ruft `osSemaphoreRelease` für FreeRTOS auf und weckt unsere blockierte Task auf, die sich dann das HSEM-Schloss sichert. Das ist Non-Blocking-Multicore-Synchronisation.

Folie

16/51

### Multicore Deadlock-Gefahr

- Ein Horrorszenario im Multicore-System:
  - Der M7 sperrt das Hardware-Schloss (HSEM).
  - Genau in diesem Moment führt ein fehlerhafter Pointer auf dem M7 zu einem Hard Fault Exception (Crash).
  - Der M7 hängt für immer in der Fehlerschleife.
  - Der M4 versucht den HSEM zu holen, wird blockiert und wartet ewig.
- Das gesamte System ist durch einen toten Kern verklemmt!

## Notizen

Aber auch Hardware-Schlösser bringen Gefahren mit sich. Stellen Sie sich ein Horrorszenario vor. Der M7 holt sich das HSEM-Schloss, um in den Shared Memory zu schreiben. In seiner Critical Section tritt ein Bug auf. Er greift auf einen Null-Pointer zu und der Cortex-M7 stürzt mit einem Hard Fault ab. Der Kern ist effektiv tot. Was passiert mit unserem HSEM? Da der M7 nie wieder den Release-Befehl ausführen wird, bleibt das Schloss für die Ewigkeit verriegelt. Der M4 auf der anderen Seite des Busses wartet auf ein Interrupt-Signal, das niemals kommen wird. Ein Fehler auf Core 1 hat das RTOS auf Core 2 vollständig lahmgelegt. Ein Multicore-Deadlock.

## Folie

17 / 51

### Die Lösung: HSEM Clear by Core ID

- Die ST-Ingenieure haben diesen Fall bedacht.
- Wenn der M4 feststellt (z. B. durch einen Heartbeat-Timeout), dass der M7 abgestürzt ist, kann er die Regeln brechen!
- Die Funktion: `HAL_HSEM_ReleaseAll(Key, CORE_ID_M7)`
- Der M4 sendet ein Notsignal an das HSEM-Modul: „Brich alle Schlösser auf, die dem M7 gehörten!“
- Der M4 kann das System danach selbstständig neustarten oder in einen sicheren Zustand überführen.

## Notizen

Zum Glück haben die Hardware-Designer von STMicroelectronics mitgedacht. Die HSEM-Hardware verfügt über einen Brecheisen-Befehl. Wenn der Cortex-M4 feststellt, dass der M7 nicht mehr reagiert – weil beispielsweise ein Heartbeat-Timer abgelaufen ist – kann der M4 das Besitzrecht überstimmen. Er ruft `HAL_HSEM_ReleaseAll` auf und übergibt einen Clear-Key sowie die Core-ID des Cortex-M7. Das weist die HSEM-Hardware an: „Schließe sofort alle 32 Schlösser auf, auf denen der Name des M7 steht!“. Damit bekommt der M4 wieder Zugriff auf alle Daten, kann die Motorsteuerung sicher herunterfahren und das Gesamtsystem geordnet neu starten.

## Folie

18 / 51

### Zusammenfassung Hardware-Semaphoren

- Normale Mutexe (Software) funktionieren nicht über OS/Core-Grenzen hinweg.
- Das HSEM-Modul löst Race Conditions auf der Ebene der Bus-Matrix (HSEM an AHB4).
- Locks vergeben die Core ID für exklusives Besitzrecht.
- Wir nutzen Interrupts statt Polling, um FreeRTOS-Tasks bei belegten Schlössern schlafen zu legen.
- Im Notfall können Schlösser eines abgestürzten Kerns aufgebrochen werden.

## Notizen

Fassen wir den ersten Block zusammen. Sie haben gelernt, dass Software-Locks bei Multicore wirkungslos sind. Wir brauchen Hardware-Schlösser, die HSEM-Register. Diese garantieren uns durch Bus-Arbitrierung absolute Atomarität. Wir können sie elegant mit FreeRTOS-Blockaden und Interrupts verbinden, um die CPU zu entlasten, und wir haben einen Notfallplan, falls ein Kern abstürzt. Dieses Wissen qualifiziert Sie für die Entwicklung sicherer, asynchroner Multicore-Systeme.

## Folie

19/51

### Was ist „Safety“ in Embedded Systems?

- Bisher haben wir uns auf „Synchronisation“ konzentriert (Fehler vermeiden).
- Was passiert aber, wenn der Fehler trotzdem auftritt?
- Safety (Funktionale Sicherheit):
  - Schutz von Menschenleben und Umwelt vor Fehlfunktionen der Software/Hardware.
  - Reguliert in Normen wie der ISO 26262 (Automotive).
- Ziel: Wenn das RTOS in einem Deadlock hängt, muss ein unabhängiges System den Stecker ziehen und neustarten!

## Notizen

Damit wechseln wir nahtlos zum zweiten großen Thema des heutigen Tages: Safety, zu Deutsch die funktionale Sicherheit. Bisher waren wir als Programmierer sehr optimistisch. Wir haben Mutexe und HSEMs gebaut und uns darauf verlassen, dass wir Fehler damit vermeiden. Aber was, wenn wir einen Fehler machen? Was, wenn wir einen Deadlock in den Code einbauen? Safety bedeutet, dass wir nicht nur versuchen, Fehler zu vermeiden, sondern dass wir Architekturen entwerfen, die uns retten, WENN der Fehler auftritt. Normen wie die ISO 26262 für Autos fordern zwingend Mechanismen, die das System notfalls neu starten, bevor Schlimmeres passiert.

## Folie

20/51

### Ursachen für System-Crashes

- Warum stürzt eine eigentlich gut getestete Software plötzlich ab?
- Interne Ursachen:
  - Deadlocks durch falsch gesetzte Mutexes.
  - Endlosschleifen (`while`) durch fehlerhafte Sensorwerte.
  - Stack Overflow im RAM.
- Externe Ursachen (Umwelteinflüsse):
  - EMI (Elektromagnetische Interferenz): Ein Funke an einem dicken Elektromotor induziert Spannung auf der Platine.
  - Bit-Flips: Kosmische Strahlung ändert physikalisch eine 0 im RAM zu einer 1 (aus

einer Schleife bis 100 wird plötzlich eine Schleife bis 65535!).

### Notizen

Warum stürzt Code, der monatelang auf dem Schreibtisch lief, im Feld plötzlich ab? Natürlich gibt es interne Software-Fehler: Der gefürchtete Stack Overflow oder ein versteckter Deadlock. Aber noch viel gefährlicher sind Umwelteinflüsse. Wenn direkt neben Ihrem Steuergerät ein 100-Kilowatt Elektromotor anläuft, schlagen elektromagnetische Pulse auf Ihre Platine ein. Das kann die CPU verwirren. In der Luft- und Raumfahrt, aber auch schon auf der Erde, gibt es Bit-Flips durch Höhenstrahlung. Ein Neutron trifft den Transistor Ihres RAM-Speichers und kippt eine 0 in eine 1. Plötzlich ist Ihr Schleifenzähler gigantisch groß, und das System friert ein. Dagegen hilft kein C-Code der Welt. Wir brauchen Hardware-Unterstützung.

### Folie

21 / 51

## Das Konzept des Hardware-Watchdogs

- Die Analogie: Der Totmannschalter in Zügen. Der Lokführer muss regelmäßig ein Pedal treten. Tut er es nicht, bremst der Zug automatisch.
- Der Watchdog (Wachhund) in der Hardware:
  - Ein eigenständiger Zähler-Chip auf der Platine (oder im STM32).
  - Der Zähler zählt unerbittlich rückwärts: 1000 → 999 → 998 ...
  - Fällt er auf 0, zieht die Hardware die RESET-Leitung der CPU! Das gesamte System startet hart neu.

### Notizen

Die Lösung dafür ist so alt wie die Elektronik selbst: Der Hardware-Watchdog. Das Konzept entspricht dem Totmannschalter aus dem Schienenverkehr. Der Lokführer muss alle paar Sekunden ein Pedal treten. Tut er es nicht, geht die Technik davon aus, dass er ohnmächtig ist, und zieht die Notbremse. Genau das ist ein Watchdog. Es ist ein dummer, aber robuster Hardware-Zähler. Er kennt nur eine Aufgabe: Er zählt gnadenlos rückwärts. Von 1000 auf 0. Und wenn er die Null erreicht, triggert er hardwareseitig den Master-Reset-Pin Ihres Mikrocontrollers. Der Chip wird zwangsweise neu gebootet.

## Die Pflicht der Software (Kicking)

- Damit das System nicht ständig neustartet, muss die Software den Watchdog „füttern“ (Kicking / Refreshing).
- Die Software sendet periodisch einen Befehl an das Hardware-Register.
- Der Befehl setzt den Zähler wieder auf seinen Maximalwert (z. B. 1000).
- Die Logik:
  - Läuft die Software fehlerfrei, wird der Zähler regelmäßig zurückgesetzt.
  - Hängt die CPU in einer Endlosschleife, wird der Refresh-Befehl nicht erreicht.
  - Der Zähler fällt auf 0 → Reset!

### Notizen

Damit dieser Wachhund unser laufendes System nicht ständig neustartet, müssen wir ihn streicheln, oder auf Englisch “kicking the dog” beziehungsweise füttern. Unsere Software muss, solange sie gesund ist, in regelmäßigen Abständen ein bestimmtes Register beschreiben. Das setzt den Rückwärtszähler wieder zurück auf seinen Startwert. Die Logik ist absolut wasserdicht: Wenn unsere Task in einen Deadlock gerät oder die CPU wegen eines Bit-Flips in den Wald springt, wird der C-Code für das Füttern nicht mehr ausgeführt. Der Zähler fällt unweigerlich auf Null und der Reset befreit uns aus der Blockade.

## Watchdog Typ 1: Independent Watchdog (IWDG)

- Der STM32H755 besitzt zwei verschiedene Watchdogs. Der erste ist der IWDG.
- Er heißt „Independent“ (Unabhängig), weil er seine eigene, isolierte Taktquelle besitzt.
- Er läuft auf dem LSI-Oszillator (Low-Speed Internal).
- Eigenschaften des LSI:
  - Frequenz:  $\approx 32$  kHz (sehr langsam, sehr unpräzise).
  - Aber: Völlig unabhängig vom Haupt-Quarz und den PLL-Schaltungen der CPU!

### Notizen

Der STM32H755 bietet uns gleich zwei verschiedene Wachhunde an. Der erste ist fürs Grobe zuständig: Der Independent Watchdog, kurz IWDG. Das Wort “Independent” ist hier Programm. Dieser Watchdog nutzt nicht den schnellen CPU-Takt. Er hat tief im Silizium einen komplett eigenen, isolierten Oszillator verbaut: den LSI (Low-Speed Internal). Dieser LSI tuckert gemütlich mit 32 Kilohertz vor sich hin. Er ist furchtbar ungenau, schwankt je nach Temperatur, hat aber eine unschlagbare Eigenschaft: Er ist physikalisch unabhängig vom Hauptquarz des Chips.

## Der Vorteil des LSI

- Warum ist diese Unabhängigkeit so wichtig?
- Wenn ein Hardware-Schaden auftritt (z. B. der Haupt-Quarz auf der Platine reißt durch Vibration ab), bleibt die gesamte CPU stehen.
- Ein normaler Timer würde stoppen.
- Der IWDG läuft auf seinem separaten 32 kHz-Takt unbeirrt weiter!
- Er erkennt, dass die tote CPU nicht mehr füttert, und zwingt das Board in den Reset.
- Der IWDG ist der Retter für Hardware- und grobe Software-Ausfälle.

### Notizen

Warum spendiert STMicroelectronics einen extra Oszillator nur für den Watchdog? Weil Safety auch Hardware-Ausfälle abdecken muss! Stellen Sie sich vor, Ihr Auto fährt über ein Schlagloch, und durch die Vibration reißt die Lötstelle des externen Takt-Quarzes ab. Die CPU bleibt sofort stehen, da sie keinen Takt mehr hat. Jeder Timer, der am Haupt-Bus hängt, stoppt ebenfalls. Ein normaler Watchdog würde einfrieren und das Auto wäre ein toter Stein. Nicht so der IWDG. Da er seinen eigenen internen Taktgeber hat, läuft sein Countdown völlig unbeeindruckt weiter. Er merkt, dass die CPU tot ist, triggert den Reset und rettet das System. Er ist die ultimative Rückversicherung gegen das totale Versagen.

## Konfiguration des IWDG

- Die Hardware des IWDG besteht aus zwei Werten, die wir im CubeMX einstellen:
  - Prescaler (Vorteiler): Teilt die 32 kHz des LSI (z. B. durch 32  $\rightarrow$  1 kHz = 1 ms/Tick).
  - Reload-Wert: Der Wert, auf den der Zähler beim „Füttern“ gesetzt wird (z. B. 500).
- Ergebnis: Das System muss zwingend alle 500 ms gefüttert werden!
- Der Code zum Füttern (Refresh) in der CubeIDE ist simpel:

```
HAL_IWDG_Refresh(&hiwdg);
```

### Notizen

Die Konfiguration des IWDG ist denkbar einfach. Wir stellen im Cube-Configurator einen Prescaler ein, um den langsamen 32-kHz-Takt auf etwas Brauchbares wie 1 kHz herunterzuteilen. Dann geben wir einen Reload-Wert vor, zum Beispiel 500. Das bedeutet, unser Watchdog hat eine Frist von 500 Millisekunden. In unserem C-Code rufen wir dann periodisch das HAL-Makro HAL\_IWDG\_Refresh auf, was den Zähler hardwareseitig immer wieder auf 500 hochschießt.

## Die Gefahr im RTOS: Wo füttern wir?

- In einem Bare-Metal-Programm rufen wir den Refresh einfach am Ende der `while (1)`-Schleife auf.
- Aber in einem Multitasking-System (FreeRTOS) laufen 10 verschiedene Tasks!
- Die absolute Kernfrage: Wo platzieren wir den `HAL_IWDG_Refresh()`-Aufruf?
- Wenn wir es falsch machen, verliert der Watchdog jeglichen Sinn!

### Notizen

Doch hier beginnt die Architektendebatte. Bei einem alten C-Programm ohne Betriebssystem ist das Füttern einfach: Wir schreiben den Befehl ans Ende unserer Super-Loop. Solange die Schleife rotiert, leben wir. Aber wir nutzen FreeRTOS! Wir haben 10 unabhängige Tasks. Wir haben Idle-Tasks, Hardware-Interrupts und SysTicks. Die Kernfrage der funktionalen Sicherheit in RTOS lautet: Wer hat die Ehre, den Watchdog zu füttern? Wenn Sie den Befehl an die falsche Stelle schreiben, wird das System füttern, obwohl das halbe Gerät bereits im Deadlock festhängt.

## Anti-Pattern: Füttern im Timer-Interrupt

- „Bauen wir uns einen Timer, der jede Sekunde feuert und den Hund füttert!“
- Warum das tödlich ist:
  - Timer-Interrupts basieren auf Hardware. Sie laufen auch dann weiter, wenn FreeRTOS komplett im Deadlock hängt!
  - Alle Tasks sind eingefroren, aber die ISR füttert stoisch den Watchdog.
  - Der Reset wird niemals ausgelöst.

```
// NIEMALS TUN!  
void HAL_TIM_PeriodElapsedCallback(  
    TIM_HandleTypeDef *htim)  
{  
    // Die Tasks sind alle deadlocked,  
    // aber wir füttern trotzdem!  
    HAL_IWDG_Refresh(&hiwdg);  
}
```

### Notizen

Lassen Sie uns mit der schlechtesten aller Ideen anfangen, dem Anti-Pattern Nummer eins: Wir bauen uns einen Hardware-Timer, lassen diesen alle 200 Millisekunden feuern und packen den Fütter-Befehl in den Interrupt-Callback. Warum ist das tödlich? Weil Hardware-Interrupts unabhängig von unseren RTOS-

Warteschlangen feuern! Nehmen wir an, Ihre wichtigste Motor-Task und Ihre Netzwerk-Task rennen beide in einen brutalen Mutex-Deadlock. Das RTOS steht. Keine Task rechnet mehr. Aber der Hardware-Timer feuert fröhlich weiter und füttert den Wachhund! Der Watchdog denkt, das System sei gesund, und löst den rettenden Reset niemals aus. Füttern Sie niemals in Hardware-Interrupts!

## Folie

28 / 51

### Pattern 1: Füttern in der Idle-Task

- Die FreeRTOS Idle-Task (Prio 0) läuft nur, wenn alle anderen Tasks „Blocked“ oder fertig sind.
- Wir können einen Hook (`vApplicationIdleHook`) nutzen, um dort zu füttern.
- Vorteil: Wenn eine Task (z. B. Prio High) in einer `while(1)`-Endlosschleife hängt, kommt die Idle-Task nie wieder ran → Watchdog-Reset!
- Nachteil: Wenn das System gewollt auf 100 % Auslastung läuft (z. B. durch viele kleine Tasks), hat die Idle-Task keine Zeit zum Füttern → ungewollter Reset, obwohl alles okay war!

## Notizen

Ein besserer Ansatz ist das Füttern in der FreeRTOS Idle-Task. Erinnern Sie sich? Sie läuft nur auf Prio 0, wenn sonst niemand etwas zu tun hat. Das ist elegant: Wenn irgendeine Task im System Amok läuft und in eine Endlosschleife gerät, wird die Idle-Task verdrängt. Sie kommt nicht mehr dran. Das Füttern stoppt, der Watchdog resettet das fehlerhafte System. Aber auch das hat einen Haken: Was, wenn Sie das System perfekt programmiert haben, aber es ist einfach wahnsinnig viel los? Die CPU läuft gewollt auf 99% Auslastung, um Netzwerkkryptographie zu berechnen. Die Idle-Task kommt kaum zu Wort, der Fütter-Befehl kommt zu spät, und das Gerät resettet, obwohl gar kein Fehler vorlag. Das nennt man einen False Positive, einen ungewollten Reset.

## Folie

29 / 51

### Pattern 2 (Best Practice): Die Watchdog-Task

- Wir bauen uns eine eigene Task nur für die Safety!
- Priorität: Die allerhöchste im System!
- Ablauf:
  1. Die Watchdog-Task fragt alle anderen Tasks ab: „Lebt ihr noch?“ (via Event-Flags oder Mailboxes).
  2. Jede Task muss regelmäßig ein „Ich lebe“-Bit setzen.
  3. Die Watchdog-Task prüft: Sind alle Bits gesetzt?
  4. Wenn ja → Füttere den IWDG.
  5. Wenn nein → Füttere nicht. Löse den Reset aus.

- So wissen wir, dass jede einzelne Task fehlerfrei läuft.

## Notizen

Die Industrie-Best-Practice ist deshalb die dedizierte Watchdog-Task. Wir legen eine eigene Task an und geben ihr die absolute Höchstpriorität. Sie läuft zum Beispiel alle 100 Millisekunden. Aber sie füttert nicht einfach blind. Sie holt sich "Lebenszeichen" von allen anderen Tasks ein. Die Motor-Task muss ein Flag setzen, die Display-Task muss ein Flag setzen. Wenn die Watchdog-Task erwacht, prüft sie alle diese Flags. Wenn auch nur eine einzige Task seit dem letzten Durchlauf ihr Flag nicht gesetzt hat, weiß die Safety-Task: "Oh, das Display hängt!". Und dann entscheidet sie aktiv, den Fütter-Befehl zu überspringen und den System-Reset auszulösen. Das ist professionelles RTOS-Safety-Design.

## Folie

30 / 51

### Die Schwäche des IWDG

- Der Independent Watchdog ist super gegen Deadlocks und Systemhänger.
- Was er nicht erkennt: Zu schnelle oder chaotische Ausführung!
- Beispiel: Ein Pointer-Bug bringt die CPU dazu, wichtige Sicherheits-Checks einfach zu überspringen. Der Code läuft rasend schnell ab.
- Da der Fütter-Befehl immer noch erreicht wird, merkt der IWDG von diesem Chaos nichts.
- Wir brauchen einen Wächter, der prüft, ob der Code exakt in der vorherberechneten Zeit abläuft.

## Notizen

Der Independent Watchdog ist hervorragend für das Grobe. Er fängt Hardwareausfälle und klassische Deadlocks ab. Aber er hat eine Schwachstelle. Was passiert, wenn die Software nicht stehenbleibt, sondern wegen eines zerschossenen Pointers wichtige Sicherheitschecks überspringt? Der Code rast durch die Main-Loop, viel schneller als normal, und feuert den Refresh-Befehl viel zu früh ab. Dem IWDG ist das egal, er ist glücklich, wenn er Futter bekommt, egal wie oft. Wir brauchen für sicherheitskritische Code-Pfade, wie die Zündung eines Airbags, eine zweite Überwachungsinstanz, die genau misst, ob die Ausführung exakt so lange gedauert hat, wie sie physikalisch hätte dauern müssen.

## Typ 2: Window Watchdog (WWDG) Einführung

- Hier greift Typ 2: Der Window Watchdog (WWDG).
- Er hängt nicht am langsamen LSI, sondern direkt am schnellen Haupt-Bus der CPU (APB1).
- Eigenschaften:
  - Taktiert extrem präzise.
  - Stoppt, wenn der Haupt-Takt (die PLL) ausfällt.
- Das Feature: Das „Fenster“-Konzept. Sie dürfen den Hund nicht nur nicht zu spät füttern, Sie dürfen ihn auch nicht zu früh füttern!

### Notizen

Hier betritt der zweite Watchdog unseres Chips die Bühne: Der Window Watchdog, kurz WWDG. Im Gegensatz zum IWDG ist er tief in das System integriert. Er hängt direkt am schnellen Haupttakt der CPU. Er ist damit genauso präzise getaktet wie Ihre CPU-Befehle, stürzt aber logischerweise mit ab, wenn der Quarz kaputtgeht. Sein absolut herausragendes Feature ist das Fenster-Konzept. Beim WWDG reicht es nicht, einfach irgendwann vor Ablauf der Frist zu füttern. Sie müssen exakt in einem definierten Zeit-Fenster füttern. Nicht zu spät, aber eben auch nicht zu früh! Als Nächstes schauen wir uns an, wie dieses Fenster aussieht — und wie wir es im Labor programmieren.

## Das Fenster-Konzept (Window Watchdog)

- Beim WWDG dürfen Sie nicht einfach „irgendwann“ füttern.
- Er definiert ein zeitliches **Fenster (Window)**.
- **Die Phasen des Zählers:**
  1. **Early Phase (Zu früh):** Der Zähler ist frisch gestartet. Wenn Sie *jetzt* füttern, geht die Hardware von einer außer Kontrolle geratenen Software aus → **RESET!**
  2. **Allowed Window (Erlaubt):** Der Zähler hat einen definierten Schwellenwert unterschritten. Nur hier ist das Füttern (`Refresh`) legal!
  3. **Late Phase (Zu spät):** Der Zähler unterschreitet `0x40` (Übergang nach `0x3F`). Die Deadline wurde verpasst → **RESET!**
- Hinweis: Der EWI feuert bei `0x40`; der Reset folgt typisch beim nächsten Tick darunter.

### Notizen

Wie funktioniert dieses Fenster-Konzept? Beim IWDG waren wir froh, wenn die Software überhaupt noch am Leben war und irgendwann vor Ablauf der Null gefüttert hat. Der Window Watchdog verlangt Präzision. Er unterteilt seinen Countdown in drei Phasen. Wenn der Countdown startet, befinden wir uns in der „Early Phase“. Die Logik dahinter: Wenn Ihr Code für die Ausführung der Berechnungen

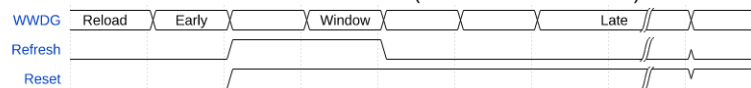
normalerweise 15 Millisekunden braucht, Sie den Watchdog aber schon nach 2 Millisekunden füttern, muss ein Programmfehler vorliegen – vielleicht hat ein kaputter Pointer die Berechnungen übersprungen! Die Hardware bestraft dieses zu frühe Füttern mit einem sofortigen Reset. Sie dürfen den Befehl nur in einem exakt definierten „Allowed Window“ aufrufen.

## Folie

33 / 51

### Visualisierung des WWDG-Fensters

- Zeitlicher Verlauf des WWDG-Countdowns (abwärts zählend):



- **Early:** Refresh zu früh → sofortiger Reset.
- **Window:** Nur hier ist Refresh erlaubt (Allowed Window).
- **Late:** Zähler unter  $0 \times 40$  → Reset.

## Notizen

Hier sehen Sie das Prinzip im WaveDrom-Diagramm. Der Zähler startet ganz links. Wenn Ihr C-Code in dieser Early Phase den Refresh-Trigger auslöst (das Signal in der Mitte geht auf 1), zieht die WWDG-Hardware sofort gnadenlos die Reset-Leitung (unterstes Signal). Das System stirbt. Wenn Sie sich brav an die Spielregeln halten und warten, bis die Hardware das „Window“ öffnet, und dann den Refresh senden, springt der Zähler einfach wieder an den Start zurück. Verpassen Sie das Fenster und der Zähler erreicht die Null, wird ebenfalls der Reset ausgelöst.

## Folie

34 / 51

### IWDG und WWDG im Doppelpack

- Der WWDG ist präzise (APB1), aber abhängig vom Haupt-Takt.
- Wenn die PLL abstürzt, friert auch der WWDG ein!
- Daher in der Automobilindustrie oft kombiniert:
  - **IWDG:** grobe Hardware-Ausfälle und Deadlocks (autarker LSI).
  - **WWDG:** präzise Überwachung von Software-Routinen und Echtzeit-Timing.
- Faustregel: Robustheit *und* Timing — nicht entweder/oder.

## Notizen

Warum brauchen wir diesen Watchdog überhaupt, wenn wir doch schon den unabhängigen IWDG haben? Weil der WWDG nicht am ungenauen 32-kHz-Oszillator hängt. Der WWDG ist fest mit dem schnellen APB1-Bus der CPU verdrahtet. Seine Takte sind absolut präzise. Er kann überwachen, ob eine zeitkritische DSP-Berechnung exakt zwischen 10 und 12 Millisekunden gedauert hat. Wenn der Code 9 oder 13 Millisekunden braucht, greift er ein. Der Preis für diese Präzision: Wenn der Hauptquarz kaputtgeht, stirbt

auch dieser Watchdog. Deshalb nutzen professionelle Systeme oft beide! Den IWDG für den absoluten Worst-Case-Hardware-Ausfall, und den WWDG für das feingranulare Software-Timing.

## Folie

35 / 51

### Warum WWDG? (Fehlerhafte Ausführung)

- Das Szenario:
  - Eine hochpriorie Task soll Sensordaten validieren (dauert 10 ms).
  - Ein Bit-Flip (kosmische Strahlung) verändert den Programmzähler (PC) oder einen Pointer.
  - Die Task überspringt die Validation und erreicht nach 1 ms das Ende.
- Beim IWDG: Die Task füttert nach 1 ms. Der IWDG sagt: „Super, du lebst!“. Die falschen Daten gehen ins System.
- Beim WWDG: Die Task füttert nach 1 ms (Early Phase). Der WWDG sagt: „Du bist zu schnell, da stimmt was nicht!“ → RESET.

## Notizen

Um den Mehrwert noch einmal greifbar zu machen: Stellen Sie sich vor, ein Neutron aus der Höhenstrahlung schlägt in Ihr RAM-Modul ein und kippt ein Bit in einer C-Schleifen-Bedingung. Ihre Validierungsfunktion, die sonst 10 Millisekunden läuft, ist nach 1 Millisekunde plötzlich „fertig“. Dem Independent Watchdog ist das egal. Er ist einfach froh, dass er seinen Refresh-Befehl bekommen hat. Er winkt den fehlerhaften Zustand durch und Ihr Auto beschleunigt, obwohl es bremsen sollte. Der Window Watchdog fängt diesen subtilen Fehler ab, weil er erkennt, dass der Refresh physikalisch unmöglich schon nach 1 Millisekunde hätte eintreffen dürfen.

## Folie

36 / 51

### Das Sicherheitsnetz: EWI

- Wenn der Watchdog unter  $0 \times 40$  fällt, zieht er hart die Reset-Leitung. Alles ist sofort tot.
- Was ist, wenn wir vorher noch Motoren sicher abschalten oder ein Crash-Log in den Flash schreiben müssen?
- Die Lösung: Der Early Wakeup Interrupt (EWI).
- Der WWDG kann so konfiguriert werden, dass er kurz vor dem Reset (bei Zählerstand  $0 \times 40$ ) den Early Wakeup Interrupt auslöst (WWDG\_IRQn, maskierbar im NVIC).
- Typisch konfigurieren wir ihn mit höchster NVIC-Priorität (Prio 0) — das ist **kein** ARM-NMI.

## Notizen

Ein harter Hardware-Reset ist brutal. Wenn die Pins des Controllers stromlos geschaltet werden, während ein großer Roboterarm gerade mit Vollgas dreht, haben Sie ein mechanisches Problem. Wir brauchen

eine Vorwarnung. Der STM32 bietet beim Window Watchdog genau das an: den EWI, den Early Wakeup Interrupt. Bei Zählerstand 0x40 feuert die Hardware den Interrupt `WWDG_IRQn` — einen normalen, maskierbaren Core-IRQ im NVIC. In der Praxis geben wir ihm oft Priorität 0 (höchste konfigurierbare Priorität), damit Applikations-ISRn ihn nicht verdrängen. Das ist fachlich etwas anderes als der ARM Non-Maskable Interrupt (NMI).

Folie

37/51

### Nutzen des EWI (Safe Shutdown)

- Der EWI feuert einige Mikrosekunden vor dem tödlichen Reset.
- In dieser speziellen ISR (`HAL_WWDG_EarlyWakeupCallback`) können wir:
  - PWM-Signale für Motoren sofort auf 0 (Safe State) ziehen.
  - Wichtige Variablen (Fehlercodes) in den Backup-SRAM retten.
  - Die rote Fehler-LED einschalten (Nucleo-H755ZI-Q: LD3 an **PB14**).
- **Wichtig:** Wir dürfen den Watchdog im EWI nicht mehr füttern! (Der Reset ist unausweichlich).

### Notizen

Dieser Early Wakeup Interrupt verschafft Ihnen auf dem Sterbebett des Mikrocontrollers noch ein paar wertvolle Mikrosekunden Lebenszeit. In der Callback-Routine dieses Interrupts dürfen Sie keine aufwändigen Berechnungen mehr starten. Ihr einziger Job hier lautet Schadensbegrenzung. Sie schreiben Nullen in die Register der Motor-PWMs, um mechanische Schäden zu verhindern. Sie kopieren vielleicht noch schnell einen Fehlercode in den batteriegestützten Backup-SRAM, damit Sie nach dem Reboot wissen, warum das System abgestürzt ist. Und ganz wichtig: Versuchen Sie nicht, in diesem EWI den Watchdog noch schnell zu füttern, um den Reset zu verhindern! Wenn Sie hier gelandet sind, ist das System korrupt. Der Reset ist die einzig richtige Konsequenz.

Folie

38/51

### HAL-API für den WWDG

- So nutzen wir den Window Watchdog in C:

```
// 1. Initialisierung (oft im CubeMX generiert)
// Prescaler, Window-Wert und Counter-Reload definieren.

// 2. Füttern in der RTOS-Task (nur im erlaubten Fenster!)
HAL_WWDG_Refresh(&hwwdg);

// 3. Notfall-Rettung (wenn EWI aktiviert)
void HAL_WWDG_EarlyWakeupCallback(WWDG_HandleTypeDef *hwwdg)
{
    // MOTOR AUS!
    Motor_SafeShutdown();
    // System stirbt in wenigen Taktzyklen...
}
```

### Notizen

Die C-API für den WWDG ist fast identisch zum IWDG. Sie haben einen Refresh-Befehl, den Sie in Ihrer überwachten Task aufrufen. Die Kunst liegt darin, das Timing des Schedulers so zu designen, dass diese C-Zeile immer exakt im erlaubten Zeitfenster der Hardware ausgeführt wird. Falls nicht, landen Sie unwiderruflich in der EarlyWakeupCallback-Funktion, Ihrem Safe-Shutdown-Code.

### Folie

39/51

## Zusammenfassung: Watchdogs

| Feature          | IWDG (Independent)                   | WWDG (Window)                               |
|------------------|--------------------------------------|---------------------------------------------|
| Taktquelle       | Separater LSI (32 kHz)               | Hauptbus APB1 (schnell)                     |
| Fehler-Szenario  | Deadlocks, Hardware-Hänger           | Zu schnelle/langsame Ausführung             |
| Refresh-Regel    | Vor Ablauf der Deadline              | Nur im erlaubten Zeit-Fenster               |
| Vorwarnung (EWI) | Nein, sofortiger Hard-Reset          | Ja (Interrupt kurz vor Reset)               |
| Einsatz im RTOS  | Watchdog-Task überwacht alle anderen | Kritische Steuer-Task überwacht sich selbst |

### Notizen

Wir fassen das Safety-Kapitel der Watchdogs zusammen. Der IWDG ist unser robuster Panzer. Er läuft immer, selbst wenn die System-Clock abstürzt. Er schützt uns vor Deadlocks, bietet aber keine Vorwarnung. Wir füttern ihn meist aus einer dedizierten, hochprioren OS-Safety-Task. Der WWDG ist unser Präzisionswerkzeug. Er prüft, ob die Takte der Ausführung exakt stimmen, und zwingt uns ins „Allowed Window“. Er gibt uns durch den EWI die Chance auf einen sauberen mechanischen Shutdown

und wird direkt in der sicherheitskritischen Task (z. B. der Motorregelung) platziert. Gibt es Fragen zu den Watchdogs, bevor wir uns dem letzten Safety-Feature, dem Speicherschutz, widmen?

Folie

40 / 51

## Speicherschutz: Die MPU (Erinnerung)

- In Vorlesung 6 haben wir die MPU (Memory Protection Unit) kennengelernt, um den Cache für den SRAM4 abzuschalten.
- Aber der eigentliche Name lautet „Protection“ (Schutz).
- Das Problem in C und RTOS:
  - Task A legt ein Array `uint8_t buffer[10]` an.
  - Ein Bug in Task A schreibt in `buffer[15]`.
  - Dieser Bereich im RAM gehört aber eigentlich Task B!
  - Task A überschreibt heimlich die Variablen von Task B → Datenkorruption ohne Fehlermeldung!

## Notizen

Damit betreten wir das finale Gebiet der System-Safety: Speicherschutz. Sie kennen die MPU, die Memory Protection Unit, bereits aus Vorlesung 6, als wir sie missbraucht haben, um den L1-Cache zu deaktivieren. Aber ihr eigentlicher Zweck ist der Schutz vor fehlerhafter Software. In C gibt es keine automatische Array-Grenzen-Überprüfung. Wenn Sie ein Array der Größe 10 anlegen und aus Versehen in den 15. Index schreiben, tut die Hardware das klaglos. In einem RTOS liegen die Stacks der Tasks oft direkt hintereinander im RAM. Task A schreibt also blind in den privaten Stack von Task B. Das Programm stürzt nicht sofort ab, aber Task B rechnet plötzlich mit völlig falschen Sensorwerten. Ein Albtraum für die funktionale Sicherheit.

Folie

41 / 51

## Der primäre Zweck der MPU

- Die MPU ist ein Hardware-Modul, das den Adressbus der CPU überwacht.
- Wir können RAM-Regionen definieren und Zugriffsrechte vergeben:
  - Read/Write (Lesen & Schreiben erlaubt).
  - Read-Only (Schreiben verboten → gut für Konstanten).
  - No Access (komplett gesperrt).
  - Execute Never (verhindert, dass Schadcode aus dem RAM ausgeführt wird).
- Ein illegaler Zugriff löst in derselben Nanosekunde einen MemManage Fault (Hardware-Interrupt) aus.

## Notizen

Um das zu verhindern, nutzen wir die MPU. Die MPU überwacht jeden einzelnen Adressaufruf der CPU, bevor er auf den Bus geht. Wir können den RAM in Regionen einteilen und Rechte vergeben. Wir können sagen: „Dieser Bereich ist Read-Only“. Versucht eine Task dorthin zu schreiben, feuert die MPU in exakt derselben Nanosekunde den MemManage Fault Interrupt ab. Die CPU wird hardwareseitig gestoppt, BEVOR das falsche Byte in den RAM geschrieben werden kann.

## Folie

42 / 51

### Use-Case 1: Null-Pointer-Protection

- Ein häufiger C-Fehler: Zugriff auf einen uninitialisierten Pointer (`NULL` = Adresse `0x00000000`).
- Normalerweise liegt bei Adresse `0x0` der Flash-Speicher. Die CPU liest bei einem Null-Pointer-Dereference also einfach den Assembler-Code und interpretiert ihn als Daten. Kein Absturz, aber falsche Berechnungen!
- Die Safety-Lösung:
  - Wir konfigurieren MPU Region 0 für die allerersten Bytes im Adressraum.
  - Wir setzen die Rechte auf „No Access“.
  - Ein Null-Pointer-Zugriff löst nun sofort einen Absturz aus, der abgefangen und geloggt werden kann!

## Notizen

Ein genialer Einsatzzweck für die MPU ist die Null-Pointer-Protection. Was passiert in C, wenn Sie vergessen, einen Zeiger zu initialisieren? Er zeigt auf Null, also auf die physikalische Speicheradresse `0x00000000`. Bei Cortex-M-Controllern beginnt dort meistens der Flash-Speicher. Wenn Ihr Code nun Daten aus diesem Pointer liest, stürzt er nicht ab! Er liest einfach den Binärcode Ihres eigenen Programms aus und interpretiert ihn als Sensorwert. Das ist fatal. Ein Safety-Ingenieur legt deshalb eine kleine MPU-Region auf die Adresse Null und deklariert sie als „No Access“. Sobald Ihr Code nun einen uninitialisierten Pointer nutzt, greift die Hardware hart ein und wirft die Fault-Exception. Sie haben den C-Bug isoliert.

## Folie

43 / 51

### Use-Case 2: Stack-Overflow Erkennung

- In FreeRTOS hat jede Task ihren eigenen Stack.
- Wenn der Stack voll ist, läuft er in den Speicherbereich der Nachbar-Task über.
- Die MPU-Lösung:
  - Das OS legt am Ende jedes Task-Stacks eine winzige MPU-Region („Guard Block“) an.
  - Diese Region wird auf „No Access“ gesetzt.

- Wenn der Stack wächst und in diesen Bereich eindringen will, löst die MPU sofort aus!

```
[ Stack Task 2 ]  
[ MPU Guard Block ] <- NO ACCESS!  
[ Stack Task 1 ]
```

- Die Fehlersuche wird trivial, da der Debugger exakt bei der C-Zeile stoppt, die den Stack gesprengt hat.

## Notizen

Noch eleganter ist die Stack-Overflow-Erkennung. FreeRTOS bietet hierfür spezielle MPU-Ports an. Wenn das Betriebssystem eine Task anlegt, setzt es ganz ans Ende des reservierten RAM-Bereichs eine unsichtbare „rote“ MPU-Grenze, einen Guard Block, mit den Rechten „No Access“. Wenn Ihre C-Funktion nun ein riesiges Array deklariert und der Stack-Pointer nach unten in diesen verbotenen Bereich wächst, schlägt die MPU zu. Der gigantische Vorteil: Die Hardware friert die CPU exakt bei der Zeile C-Code ein, die den Stack gesprengt hat. Ohne MPU würde das System vielleicht erst Minuten später abstürzen, weil Task 2 korrupte Daten findet.

## Folie

44 / 51

### Privileged vs. User Mode

- Wie schützt sich das OS vor den eigenen Tasks?
- Die ARM-Architektur kennt zwei Betriebsmodi:
  - **Privileged Mode (Kernel):** Darf alles, darf alle MPU-Regionen ändern.
  - **User Mode (Task):** Ist eingeschränkt, darf nur auf den eigenen RAM zugreifen.
- In sicherheitskritischen Anwendungen (wie AUTOSAR oder FreeRTOS-MPU):
  - Das Betriebssystem läuft „Privileged“.
  - Alle User-Tasks laufen im „User Mode“ und sind in ihren eigenen MPU-Sandkästen eingesperrt.
- Eine fehlerhafte Task crasht durch die MPU nur sich selbst, kann aber das OS nicht zerstören!

## Notizen

Um dieses Sicherheitsnetz perfekt zu machen, stützt man sich auf die Privilege-Level der ARM-Hardware. Normalerweise läuft Ihr gesamter Code im Kernel-Modus, dem „Privileged Mode“. Er darf alles. In der Luftfahrt oder im modernen Automotive-Bereich nutzt man stattdessen OS-Ports wie FreeRTOS-MPU. Hier läuft nur noch der OS-Kernel (wie PendSV und SysTick) im Privileged Mode. Ihre selbst geschriebenen Tasks werden von der Hardware in den sogenannten User Mode gezwungen. Sie können keine MPU-Regeln mehr ändern und sind in ihrem Sandkasten eingesperrt. Wenn Ihre Task Amok läuft, reißt sie sich selbst in den Abgrund, aber der RTOS-Kern läuft unbeeindruckt weiter und kann eine Fehlerbehandlung einleiten.

## Zusammenfassung MPU & Safety

- Safety-Mechanismen schützen Menschen vor Software-Bugs.
- Der IWDG läuft autark und schützt vor System-Deadlocks.
- Der WWDG überwacht das präzise Timing der Softwareausführung (EWI für Safe-Shutdown).
- Die MPU isoliert Tasks physikalisch voneinander und verhindert Silent-Data-Corruption (z. B. durch Stack Overflows).

### Notizen

Fassen wir den Safety-Block zusammen. Wenn wir Systeme bauen, bei denen Fehler gefährlich werden, müssen wir Redundanzen auf Siliziumebene nutzen. Der autarke IWDG rettet uns vor Totalausfällen. Der hochpräzise Window Watchdog stellt sicher, dass unsere Software nicht zu früh und nicht zu spät antwortet. Die Memory Protection Unit sichert unseren Arbeitsspeicher hardwareseitig gegen korrupte Pointer ab. Wenn Sie diese Werkzeuge beherrschen, bauen Sie Systeme auf Industrie-Niveau.

## Laborvorbereitung: Multicore-Safety

- Ziel für heute Nachmittag:
  - Wir bauen das Multicore-Ping-Pong aus Termin 6 um.
  - Wir entfernen das asymmetrische Ping-Pong-Protokoll und ersetzen es durch robuste Hardware-Semaphoren (HSEM).
  - Anschließend überwachen wir den RTOS-Kern mit dem WWDG.
- **Schritt 1: CubeMX Dual-Core Setup**
  - Projekt von Woche 6 öffnen.
  - Unter System Core → HSEM aktivieren.
  - Weisen Sie das HSEM-Modul beiden Kernen (Cortex-M7 und M4) zu!

### Notizen

Kommen wir zur Laboraufgabe für heute. Wir bauen unsere Multicore-Kommunikation von Woche 6 um. Wir werfen die Flags „m7\_ready“ und „m4\_ready“ komplett weg. Wir machen das System brutal asynchron und zwingen beide Kerne, exakt denselben Wert im SRAM4 hochzuzählen. Damit das nicht in der Zerstörung endet, müssen Sie im Cube-Configurator das HSEM-Modul aktivieren. Achten Sie darauf, den Haken bei beiden Kernen zu setzen, damit STMicroelectronics den C-Code für beide Kerne generiert.

## Schritt 2: Der asynchrone Multicore-Zugriff

- Gehen Sie in den Code beider Kerne (CM7/main.c und CM4/.../freertos.c).
- Ersetzen Sie die Logik durch eine schnelle Endlosschleife, die auf den Payload zugreift:

```
// Sowohl M7 als auch M4 führen dies asynchron aus!
// Labor-Kompromiss: Polling + osDelay(1) statt HSEM-IRQ-Pfad (T
for(;;) {
    // 1. Hardware-Schloss (ID 0) anfordern
    while (HAL_HSEM_FastTake(0) != HAL_OK) {
        osDelay(1); // Nur auf dem M4 (M7: leeres Polling)
    }

    // 2. --- CRITICAL SECTION ---
    sd.payload_value++;
    // -----

    // 3. Hardware-Schloss freigeben (Process ID 0)
    HAL_HSEM_Release(0, 0);
}
```

### Notizen

Im C-Code sieht das dann so aus. Beide Kerne rufen diese Schleife auf. Zuerst fordern wir das Schloss Nummer 0 per FastTake an. Da FastTake sofort zurückkehrt, packen wir es auf dem Cortex-M4 in eine While-Schleife mit einem osDelay(1). So gibt die FreeRTOS-Task die CPU brav ab, solange der M7 das Schloss hat. Sobald wir das „HAL\_OK“ bekommen, sind wir sicher. Wir addieren unseren Zähler im SRAM4 und geben das Schloss mit HAL\_HSEM\_Release wieder frei.

## Schritt 3: Den WWDG integrieren

- Öffnen Sie CubeMX und aktivieren Sie den WWDG für den Cortex-M4.
- Konfiguration:
  - WWDG clock prescaler: 8 (oder passen Sie es an Ihren Takt an).
  - Window value: 80 (Erlaubtes Fenster beginnt bei Zählerstand 80).
  - Free-running downcounter value: 120 (Startwert).
- Aktivieren Sie den Early Wakeup Interrupt (EWI) im NVIC-Reiter!
- Code generieren.

## Notizen

Wenn die Hardware-Schlösser funktionieren und Ihr Terminal zeigt, dass der Zähler fehlerfrei und ohne Verluste in die Höhe schnellte, bauen wir das Safety-Netz ein. Aktivieren Sie den Window Watchdog für den Cortex-M4. Konfigurieren Sie die Zählerwerte so, dass wir ein solides Fenster haben, zum Beispiel Start bei 120, Fenster öffnet bei 80, Tod bei 0. Ganz wichtig: Aktivieren Sie den Early Wakeup Interrupt, den EWI, im NVIC, damit wir im Notfall reagieren können.

## Folie

49/51

### Schritt 4: Den Wachhund füttern & Crashen

- In der FreeRTOS-Task des M4 füttern wir nun periodisch:

```
void StartDefaultTask(void *argument) {
    for(;;) {
        // ... HSEM Logik ...

        // Wir warten exakt so lange, dass wir im "Allowed Window" 1
        osDelay(15);

        // FÜTTERN!
        HAL_WWDG_Refresh(&hwwdg);
    }
}
```

- **Crash-Test 1:** Ändern Sie das Delay auf `osDelay(50)`.
  - → Zähler unter `0x40` (Late). Das Board stirbt und blinkt kurz neu.
- **Crash-Test 2:** Ändern Sie das Delay auf `osDelay(5)`.
  - → Wir füttern in der Early-Phase. Das Board stürzt sofort ab!

## Notizen

Im C-Code müssen wir das System nun haargenau kalibrieren. Fügen Sie in die Task ein `osDelay` ein, das dafür sorgt, dass Ihr Refresh-Befehl immer exakt im grünen Bereich, dem Allowed Window, landet. Das System läuft stabil. Danach prüfen Sie, ob Ihre Safety-Mechanismen wirklich funktionieren! Verlängern Sie das Delay auf 50, flashen Sie, und beobachten Sie den Reset, weil Sie zu spät füttern. Verkürzen Sie das Delay auf 5, flashen Sie, und beobachten Sie den Absturz durch die Early-Phase.

## Schritt 5: Den Safe-Shutdown implementieren

- Fügen Sie in der `main.c` des CM4 den Callback ein:

```
void HAL_WWDG_EarlyWakeupCallback(WWDG_HandleTypeDef *hwwdg)
{
    // Die letzten Mikrosekunden vor dem Reset!
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_14, GPIO_PIN_SET); // LD3

    // HIER NIEMALS REFRESH AUFRUFEN!
    // System bootet gleich hart neu.
}
```

- Wenn Sie den Crash-Test (z. B. Late-Refresh) nun ausführen, blitzt die rote LD3 (PB14) auf dem Board extrem kurz auf, bevor der Reset zuschlägt.
- Sie haben den EWI live gesehen!

### Notizen

Als Krönung implementieren wir die letzte Notfallmaßnahme. Überschreiben Sie in der `main.c` den Callback für das Early Wakeup. Schalten Sie dort zur Verifikation die rote LD3 (PB14) ein — PE1 ist auf dem Nucleo-H755ZI-Q die gelbe LD2 — oder ziehen Sie einen Pin auf HIGH, den Sie mit dem Oszilloskop messen können. Wenn Sie nun den Fehlerfall im Code provozieren, werden Sie sehen, wie die Hardware unbarmherzig eingreift, Ihre Callback-Routine für wenige Taktzyklen aufruft und das gesamte Chip-Design neu bootet.

## Die Essenz des Tages

- Sie bauen heute Nachmittag keine Labor-Software mehr.
- Sie bauen eine ausfallsichere Serien-Architektur.
- Asynchrone Multicore-Zugriffe sind durch HSEM physikalisch abgeriegelt.
- Ein unvorhergesehener Software-Hänger auf dem M4 wird durch den WWDG erkannt.
- Das System versetzt sich vor dem Absturz über den EWI in einen sicheren Zustand (Motor aus).
- Viel Erfolg im Labor!

### Notizen

Damit entlasse ich Sie in das heutige Labor. Was Sie heute programmieren, ist keine akademische Fingerübung mehr. Sie bauen eine Architektur, die Sie exakt so in ein Automotive-Steuergerät für die Serie flashen könnten. Sie verriegeln Speicherzugriffe auf der untersten Silizium-Ebene, und Sie sichern

das Echtzeit-Verhalten Ihres C-Codes mit dem stahlharten Zeit-Fenster des Window-Watchdogs ab. Das ist Embedded Engineering auf höchstem Niveau. Viel Erfolg dabei!